Test Document

Test-01

Date: 18 April 2026

Allies Connect

Molly Calhoun, Tarik Davis, Ryan Hanrahan,

Takeshia Banks, David Castro

Allies Way

Dr. N. Laughlin, Executive Director

1. Testing Strategy

1.1 Overall strategy

Our testing strategy includes two types of testing: unit tests and integration tests. Unit tests confirm that specific functions, methods, or classes work. Integration tests confirm that the requirements for the software are actually functional.

We opted not to invest time in system testing because we are deploying to a single server with a fairly simple architecture. We had limited time in which to accomplish our goals, and determined that other facets of the application were higher priority.

Similarly, with a short development timeline, and with each member of the team working in fairly discrete, siloed areas of the code, we decided not to implement regression testing. Unit tests can work as a form of regression testing by their nature. But developing specific regression tests for functional changes didn't have enough value to warrant the time that would be required.

Our tests were all developed by one of the members, David Castro, with the assistance of Gemini in Antigravity.

To perform unit tests, the development team navigates to the `/allies-connect` (front-end) and `/backend` (back-end) directories and runs `npm test`. To perform the integration tests, team members navigate to the `/allies-connect` directory and runs `npm run cypress:run`.

Team members always run both unit tests and integration tests before submitting pull requests.

1.2 Test Selection

Unit testing strategy (white-box). Our unit testing strategy involves picking out the specific functions that are most likely to be modified in the future. This ensures that the original contract continues to be fulfilled, and deviations are quickly identified and addressed.

Integration testing strategy (black-box). Our integration testing strategy involves picking critical user journeys to test. This ensures that the main functions continue to work as expected. We chose to use live database testing rather than a mocked database, which caused some issues with one function that requires a new, valid EIN each time it is run. We may flip that back to using a mocked database in the future to resolve this issue. You can see details about this in the section later in this document that lists the integration tests.

1.3 Adequacy Criterion

Because of the brief timeline for this project, we opted not to use automated tools like coverage plugins. Instead, we are relying on qualitative assessment methods. For unit testing, manually determining what code was most-likely to experience modification was more important than achieving a target percentage of coverage.

For integration testing, determining which user journeys were the most used and most critical is what guided our decisions. In a quasi-regression test, when one of the team members found functionality that they believed had worked before but then stopped working, we developed an integration test to confirm that it continues working going forward.

If this project continued beyond its initial short MVP cycle, we would switch to a more structured approach to ensure all forms of testing were included, and that all methods and all use cases were under test.

1.4 Bug Tracking

Our bug-tracking is pretty informal. Our main communication method is through our Discord server. Our bug reporting tended to not be complicated enough to require using the issues feature in GitHub, though that was available to us if we needed it.

1.5 Technology and Tools

We used the following tools for testing the application:

- Front-end unit testing: Jest and React Testing Library
- Back-end unit testing: Jest and Supertest
- End-to-end integration tests: Cypress

2. Test Cases and Test Results

Unit tests

To run unit tests, run `npm test` from the `/allies-connect` and `/backend` directories.

Table 1. Front-end unit tests

Test Case	Purpose	Expected result	Actual result	Pass/Fail
App: renders main headline	Verifies the root component loads.	Find "Georgia's Community Resource" text.	Text found in document.	Pass
Navbar: unauthenticated links	Checks guest navigation.	Visibility of "Login" and "Register" buttons.	Both buttons confirmed.	Pass
Navbar: dashboard link	Checks authenticated navigation.	Displays "Hello, John" and a "Dashboard" button.	Greeting and button found.	Pass
Home: hero copy	Verifies Landing Page branding.	Headline "Georgia's Community Resource" is present.	Headline confirmed.	Pass

Login: renders form	Checks login UI structure.	Presence of Username and Password inputs.	Inputs found via placeholder.	Pass
Login: updates values	Tests form interactivity.	Input fields reflect text typed by the user.	Values match typed input.	Pass
Register: account tabs	Verifies registration options.	"Volunteer" and "Organization" tabs are available.	Both tabs found by role.	Pass

Table 2. Back-end unit tests

Test Case	Purpose	Expected result	Actual result	Pass/Fail
Auth: Register	Verify new user creation.	User ID 5 returned; "successfully" msg.	Match.	Pass
Auth: Login	Verify credential matching.	Returns user roles and status 200.	Match.	Pass
Auth: Forgot Password	Test reset link generation.	200 OK; avoids leaking existence.	Match.	Pass
Events: List/Detail	Fetch public events.	Array of events or specific ID object.	Match.	Pass
Events: Create	Verify event + location insert.	Event ID returned; 201 Created.	Match.	Pass
Resources: List/Detail	Fetch service providers.	Returns name/address/hours object.	Match.	Pass
Resources: Create	Test provider resource entry.	201 Created; Audit log triggered.	Match.	Pass
Orgs: EIN Verify	External ProPublica API check.	Valid data for 9-digit EIN.	Match.	Pass
Orgs: Invite	Send staff invites.	201 Created; Secure link generated.	Match.	Pass

Volunteers: Signups	Manage shift registrations.	Prevents double-signup or overfilling.	Match.	Pass
Volunteers: Hours	Track completed service.	Array of hours worked per resource.	Match.	Pass
Admin: Approval	Manage pending orgs.	Changes provider status to 'active'.	Match.	Pass
Admin: Audit Logs	Track sensitive actions.	Returns list of logged admin activity.	Match.	Pass
RateLimit Middle.	Protect API from spam.	Returns 429 when max requests hit.	Match.	Pass

Integration tests

To run the integration tests, run `npm run cypress:run` from the `/backend` directory.

Table 3. Integration tests

Test Case	Purpose	Expected result	Actual result	Pass/Fail
admin_approve_org	Admin review and approval flow.	"Approved successfully" alert.	Approved & table refreshed.	Pass
map_distance_filter	Verify radius-based pin filtering.	"Far" events hidden; "Close" visible.	"Close Event" marker not found.	Fail
organization_registration	New provider signup flow.	"Registration successful" alert.	"An error occurred..." error.	Fail
user_attend_event	Guest event attendance.	RSVP confirmation message.	Success message displayed.	Pass
volunteer_contact_info	Update profile contact details.	Values persist in profile modal.	Profile updated successfully.	Pass

volunteer_edit_ avail	Manage weekly shift hours.	Availability saves and persists.	Availability grid updated.	Pass
volunteer_ manage_orgs	Staff organization list.	Displays orgs where user has roles.	List loaded correctly.	Pass
volunteer _review_signups	View signup history.	Loads list of joined shifts.	History table populated.	Pass
volunteer_signup	Join a volunteer shift.	"Signed up successfully" message.	Signup created successfully.	Pass
Volunteer_ unavail_dates	Set specific blackout dates.	Dates appear in unavailable list.	Dates persisted in UI.	Pass

Note: The two failing integration tests (at the time this document is being written) are known issues. One is due to functionality not being fully implemented, and the other due to an incompatibility between how Google Maps renders content and how the test is evaluating program state.